



REPORTE DE PRÁCTICA: SEGG- U2-P4H-1

Valeria Hernández Cortés

Grupo IDGS17

Introducción

El monitoreo y el registro de eventos son elementos muy importantes durante el desarrollo y mantenimiento de cualquier aplicación, ya que permiten conocer el comportamiento del sistema, detectar errores y facilitar el análisis de incidentes. En la práctica anterior se implementó un sistema de logging estructurado en VulnerableApp utilizando Serilog, configurando la generación de registros de forma simultánea en la consola, archivos de texto y la plataforma Seq, lo que permitió contar con un historial detallado de las actividades realizadas por la aplicación.

En esta práctica se busca ampliar esa solución incorporando una plataforma de observabilidad basada en Grafana, Loki y Promtail, aprovechando la infraestructura existente sin realizar cambios en el código de VulnerableApp. Para ello, la aplicación continuará ejecutándose de manera local mientras que los servicios de observabilidad se ejecutarán en contenedores Docker. De esta forma, los archivos de log generados por la aplicación serán recolectados por Promtail, almacenados en Loki y visualizados desde Grafana, permitiendo centralizar la información y realizar consultas más eficientes sobre los eventos registrados.

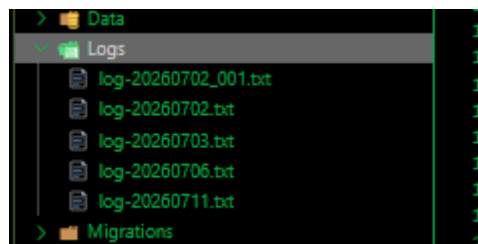
Desarrollo de la práctica

Verificación de la solución existente

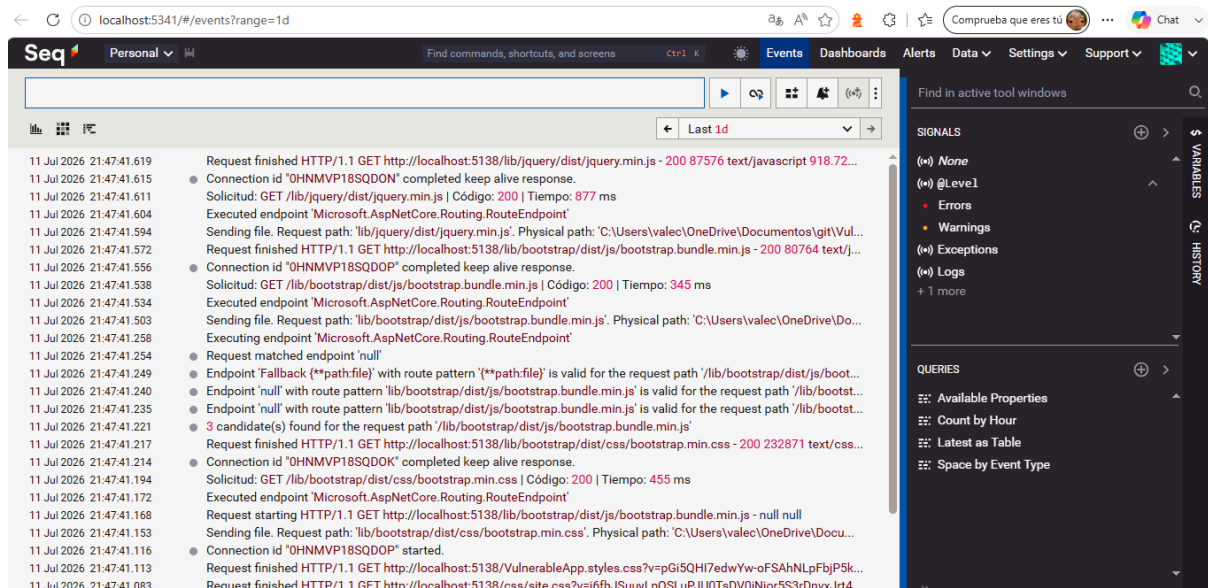
Antes de inicializar la infraestructura de contenedores, se procedió a compilar y ejecutar VulnerableApp en el entorno local . Se interactuó con la interfaz para generar tráfico y se comprobó exitosamente que los registros seguían canalizándose a las tres salidas previamente configuradas: la consola local, el directorio de logs en disco y el dashboard de Seq.



Evidencia 1: VulnerableApp generando registros



Evidencia 2: Archivos de la carpeta Logs



Evidencia 3: Eventos registrados en Seq

Investigación de la infraestructura de observabilidad

Una vez validada la solución existente, se investigó el funcionamiento de las herramientas que conforman la plataforma de observabilidad y el papel que desempeña cada una dentro de la arquitectura implementada.

Grafana es la herramienta encargada de visualizar la información mediante paneles y consultas, permitiendo analizar los registros de forma más clara y organizada.

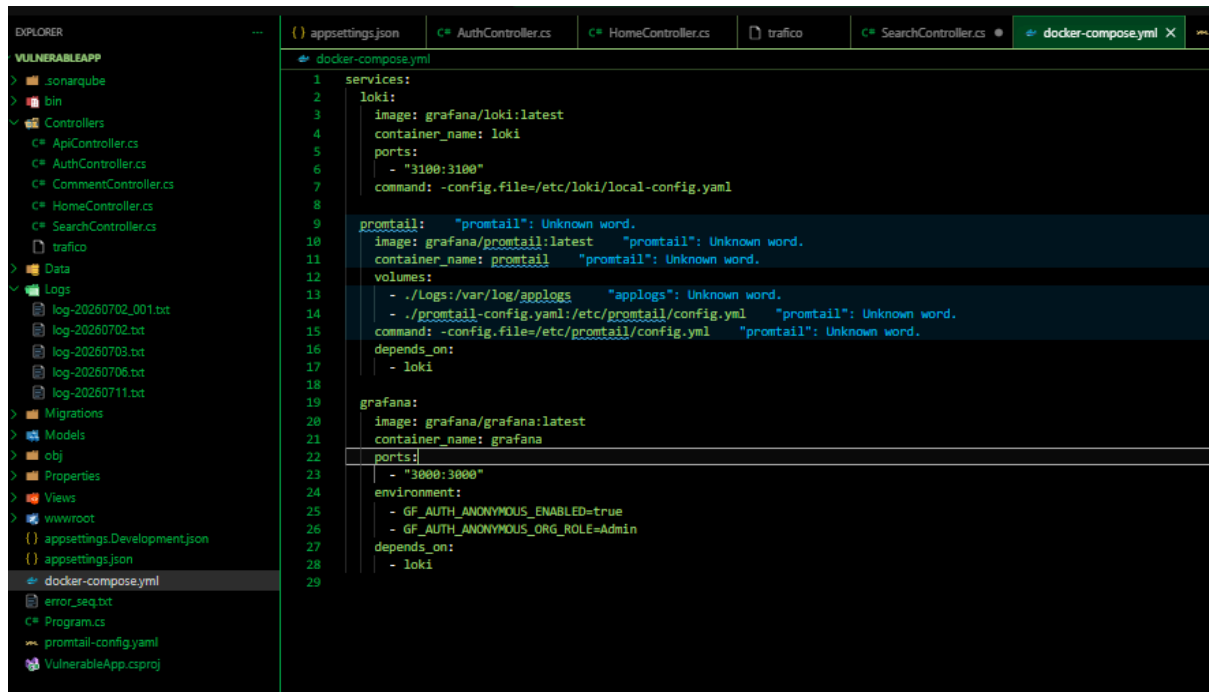
Loki es el servicio responsable de almacenar los logs enviados por Promtail. Su principal característica es que organiza la información utilizando etiquetas (labels), lo que facilita realizar consultas rápidas sin necesidad de indexar completamente el contenido de cada registro.

Promtail funciona como un agente recolector de logs. En esta práctica se configuró para monitorear los archivos generados por Serilog dentro de la carpeta Logs, enviando automáticamente cada nuevo registro hacia Loki para que posteriormente pudiera visualizarse desde Grafana.

Integración mediante Docker Compose

Para desplegar la infraestructura de observabilidad se actualizó el archivo docker-compose.yml, agregando únicamente los servicios correspondientes a Grafana, Loki y Promtail, tal como lo requería la práctica. La aplicación VulnerableApp continuó ejecutándose de forma local y no fue incorporada como un contenedor de Docker.

Durante la configuración se definieron los puertos necesarios para acceder a cada servicio, así como los volúmenes y la red interna que permite la comunicación entre los contenedores. De esta manera, Grafana puede consultar la información almacenada en Loki y Promtail puede enviar los registros que obtiene desde la carpeta de logs compartida.



```
1 services:
2   loki:
3     image: grafana/loki:latest
4     container_name: loki
5     ports:
6       - "3100:3100"
7     command: -config.file=/etc/loki/local-config.yaml
8
9   promtail:
10     image: grafana/promtail:latest
11     container_name: promtail
12     volumes:
13       - ./Logs/var/log/applogs:/etc/promtail/config.yaml
14     command: -config.file=/etc/promtail/config.yaml
15     depends_on:
16       - loki
17
18   grafana:
19     image: grafana/grafana:latest
20     container_name: grafana
21     ports:
22       - "3000:3000"
23     environment:
24       - GF_AUTH_ANONYMOUS_ENABLED=true
25       - GF_AUTH_ANONYMOUS_ORG_ROLE=Admin
26     depends_on:
27       - loki
```

Evidencia 4. Archivo docker-compose.yml actualizado

Configuración del Bind Mount

Como Promtail se ejecuta dentro de un contenedor de Docker y VulnerableApp genera los archivos de log directamente en el equipo local, fue necesario compartir la carpeta Logs mediante un Bind Mount. Esto permite que el contenedor tenga acceso a los archivos que se generan en tiempo real sin necesidad de copiarlos o modificar la aplicación.

La carpeta Logs del proyecto se enlazó con una carpeta interna del contenedor donde Promtail realiza el monitoreo de los archivos de registro.

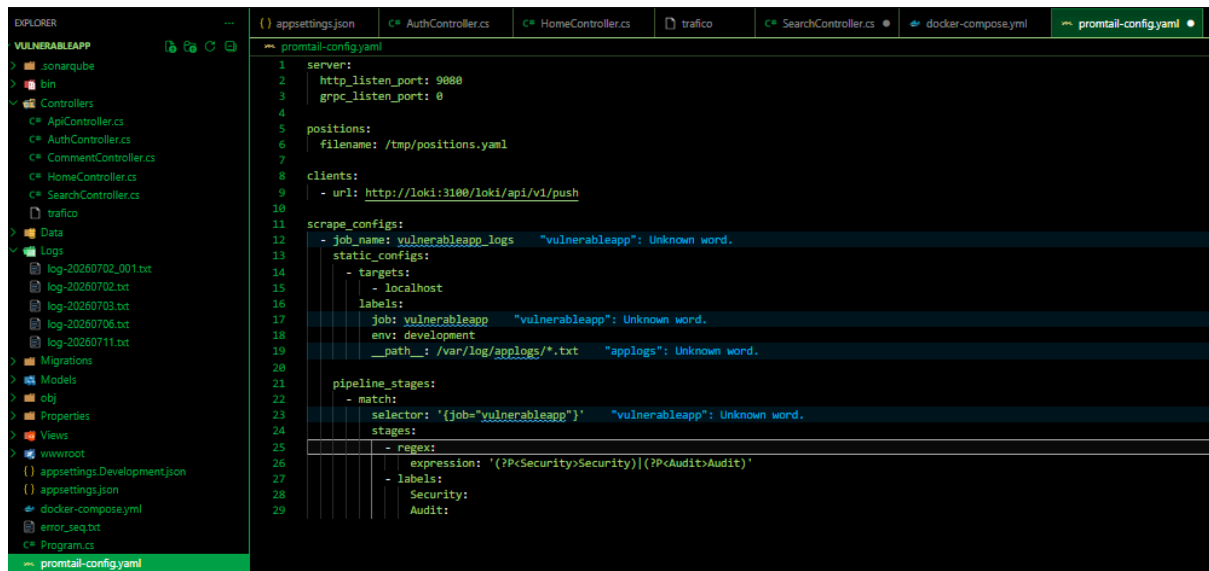
Configuración de Promtail

Después de compartir la carpeta de logs, se configuró el archivo promtail-config.yaml para indicar a Promtail dónde debía buscar los registros y cómo enviarlos a Loki.

Dentro de la configuración se definieron los siguientes elementos:

1. scrape_configs: Se creó una tarea para monitorear los archivos de log generados por VulnerableApp.

2. path: Se indicó la ruta `/var/log/applogs/*.txt`, donde Promtail buscará todos los archivos de registro.
3. positions.yaml: Archivo utilizado por Promtail para recordar hasta qué línea ha leído cada archivo, evitando que se vuelvan a enviar registros ya procesados cuando el contenedor se reinicia.
4. labels: Se agregaron etiquetas para identificar el origen de los registros y facilitar su búsqueda posteriormente desde Grafana.



```
1 server:
2   http_listen_port: 9090
3   grpc_listen_port: 0
4
5 positions:
6   filename: /tmp/positions.yaml
7
8 clients:
9   - url: http://loki:3100/loki/api/v1/push
10
11 scrape_configs:
12   - job_name: vulnerableapp_logs "vulnerableapp": Unknown word.
13     static_configs:
14       - targets:
15         - localhost
16       labels:
17         job: vulnerableapp "vulnerableapp": Unknown word.
18         env: development
19         __path__: /var/log/applogs/*.txt "applogs": Unknown word.
20
21     pipeline_stages:
22       - match:
23         selector: '{job="vulnerableapp"}' "vulnerableapp": Unknown word.
24         stages:
25           - regex:
26             expression: '(?P<Security>Security)(?P<Audit>Audit)'
27             labels:
28               Security:
29               Audit:
```

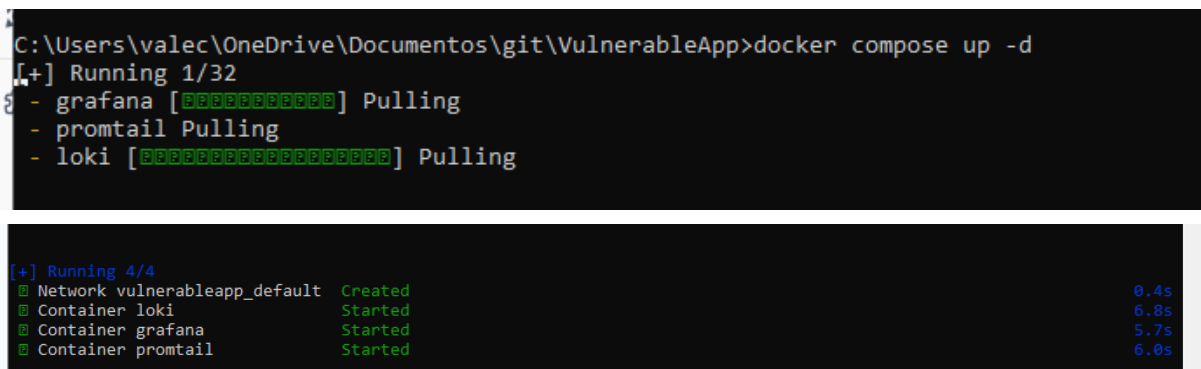
Evidencia 5. Archivo de configuración de Promtail

Despliegue de la infraestructura

Una vez terminada la configuración, se inició la infraestructura ejecutando el comando:

`docker compose up -d`

Con este comando se descargaron las imágenes necesarias y se levantaron los contenedores de Grafana, Loki y Promtail en segundo plano. Posteriormente se verificó que todos estuvieran funcionando correctamente mediante el comando `docker ps`.



```
C:\Users\valec\OneDrive\Documentos\git\VulnerableApp>docker compose up -d
[+] Running 1/32
- grafana [XXXXXXXXXXXX] Pulling
- promtail Pulling
- loki [XXXXXXXXXXXXXXXXXXXX] Pulling

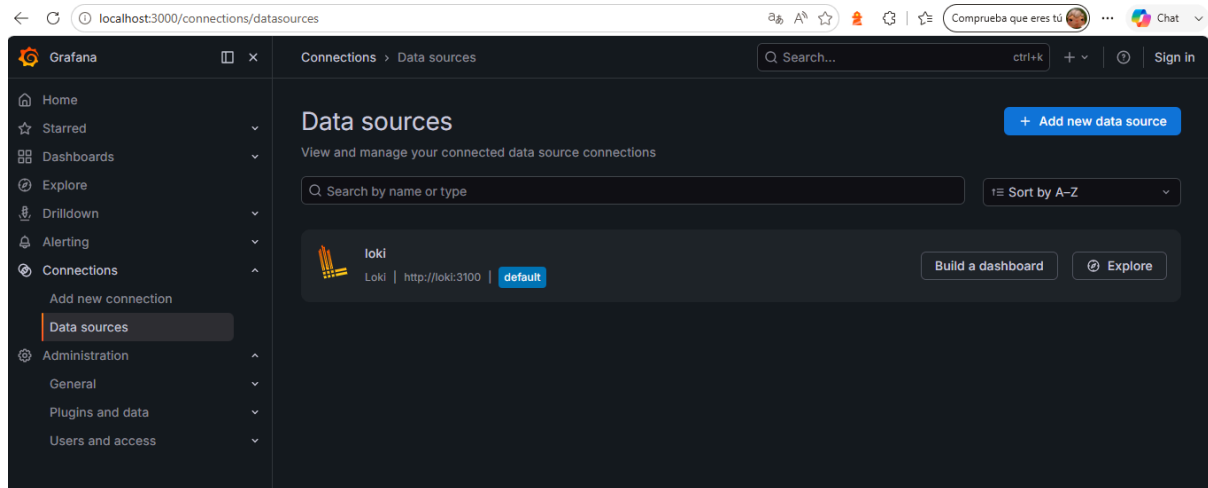
[+] Running 4/4
- Network vulnerableapp_default Created 0.4s
- Container loki Started 6.8s
- Container grafana Started 5.7s
- Container promtail Started 6.0s
```

Evidencia 6. Contenedores en ejecución

Configuración de Grafana

Después de iniciar los contenedores, se ingresó a Grafana desde el navegador utilizando la dirección `http://localhost:3000`.

Dentro de la sección Data Sources se agregó Loki como fuente de datos, indicando la dirección interna del contenedor (`http://loki:3100`). Finalmente se realizó la prueba de conexión, obteniendo una conexión exitosa.



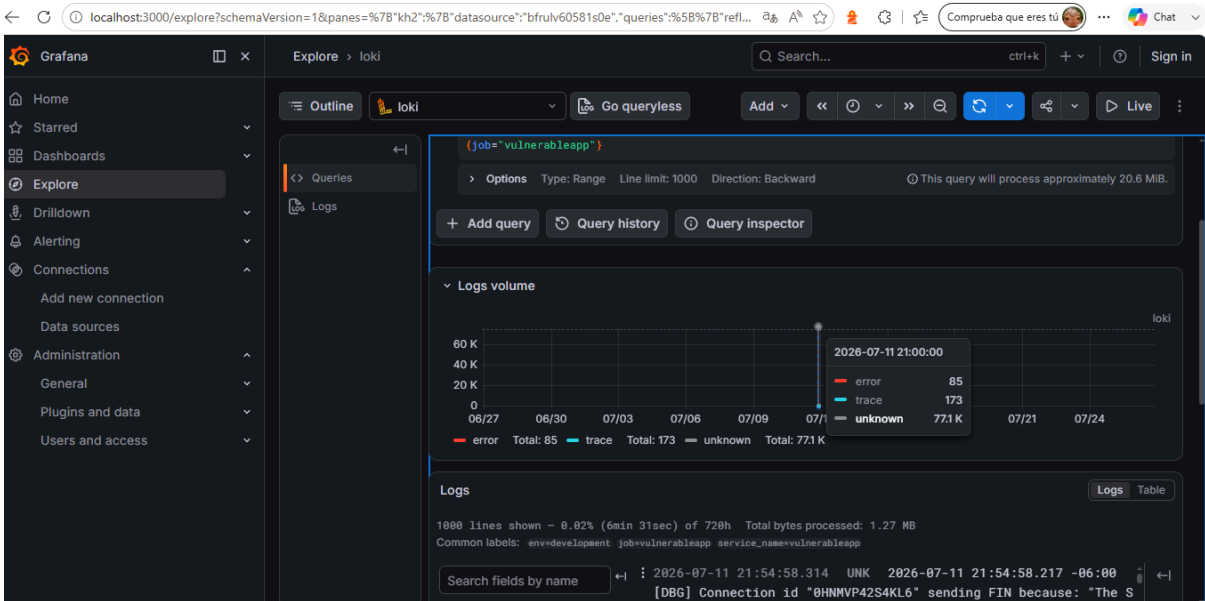
Evidencia 7. Grafana con Loki configurado

Generación de actividad

Para comprobar el funcionamiento de toda la plataforma, se ejecutó nuevamente VulnerableApp y se realizaron diferentes acciones dentro de la aplicación, como inicios de sesión, operaciones CRUD y algunas acciones que generaron advertencias y errores. Con ello se produjeron nuevos registros que posteriormente fueron enviados a Loki por medio de Promtail.

Validación de la observabilidad

Finalmente, desde la opción Explore de Grafana se realizaron consultas para verificar que los registros estuvieran llegando correctamente a Loki. Se confirmó que los eventos mostrados en Grafana coincidían con los registros almacenados en los archivos de log y con los eventos visualizados previamente en Seq.



Evidencia 8. Consulta filtrada mediante labels

Análisis y Conclusiones

Comparativa entre Seq y Grafana + Loki

Característica	Seq	Grafana + Loki
Búsqueda textual	Permite buscar fácilmente registros y propiedades.	Utiliza LogQL y etiquetas para realizar consultas más específicas.
Dashboards	Cuenta con opciones básicas de visualización.	Permite crear paneles totalmente personalizables.
Alertas	Tiene funciones limitadas para alertas.	Permite configurar alertas de forma más completa.
Consultas	Basadas en las propiedades registradas por Serilog.	Basadas en etiquetas (labels) y consultas LogQL.
Visualización	Enfocado principalmente en el análisis de logs.	Permite visualizar logs, métricas y otros datos en un mismo lugar.
Uso principal	Muy útil durante el desarrollo y las pruebas.	Ideal para monitoreo y observabilidad en producción.

Durante la práctica se observó que Seq es una herramienta muy práctica para revisar rápidamente los registros generados por una aplicación, especialmente durante el desarrollo. Por otro lado, Grafana junto con Loki ofrece una solución más completa para

monitorear aplicaciones, ya que permite organizar los registros mediante etiquetas, realizar consultas más avanzadas y crear paneles de visualización. Esto facilita el análisis de la información cuando la cantidad de registros aumenta o cuando es necesario monitorear varios servicios al mismo tiempo.

Configuración de labels

Para diferenciar los distintos tipos de registros, se agregaron labels dentro de la configuración de Promtail. Estas etiquetas permiten identificar más fácilmente el origen o tipo de cada evento y posteriormente realizar consultas específicas desde Grafana.

Se agregaron etiquetas dentro del archivo `promtail-config.yaml` para identificar los registros enviados a Loki. Además, se configuró la lectura de los archivos de la carpeta `Logs` para que Promtail enviara automáticamente cada nuevo evento.

¿Cómo utiliza Loki las labels? Loki organiza los registros utilizando las labels como identificadores. Gracias a ellas puede clasificar los eventos sin necesidad de analizar completamente el contenido de cada archivo, lo que hace más rápidas las búsquedas.

¿Qué ventajas ofrecen para realizar consultas con LogQL? Las labels permiten filtrar únicamente los registros que interesan, por ejemplo los relacionados con auditoría o seguridad. Esto hace que las consultas sean más rápidas y facilita localizar información específica cuando existen grandes cantidades de eventos registrados.

Resultados obtenidos

Al finalizar la práctica se logró integrar correctamente Grafana, Loki y Promtail con la aplicación existente, sin realizar cambios en el código de `VulnerableApp`. Los registros generados por `Serilog` fueron leídos por Promtail, enviados a Loki y visualizados desde Grafana, comprobando que toda la infraestructura de observabilidad funcionó correctamente. Además, la aplicación continuó enviando los mismos registros a Seq, permitiendo comparar ambas herramientas sin afectar el funcionamiento del sistema.

¿Qué ventajas ofrece incorporar una plataforma de observabilidad sin modificar la aplicación existente?

Una de las principales ventajas es que no fue necesario modificar el código de la aplicación para agregar nuevas herramientas de monitoreo. Esto facilita mantener el sistema funcionando mientras se agregan nuevas soluciones de observabilidad. Además, permite

actualizar o cambiar la infraestructura de monitoreo sin afectar la lógica de negocio, haciendo que el sistema sea más fácil de mantener y escalar en un entorno de producción.